Detailed Note on the Tucker Rank-Selection Algorithm for ROI10 Tensor Regression

March 25, 2026

1 Purpose

This note documents, in detail, the tensor rank-selection algorithm implemented in `H:\HBN-EEG\code\run_roi1`
The goal is to explain:

- how the EEG ROI-band summaries are converted into a $10 \times 8 \times 2$ tensor covariate,
- how 5-fold cross-validation is performed over the Tucker rank grid,
- how the fitting routine `fit_tucker_tensor_regression.m` works internally,
- how the selected rank is determined, and
- what the current default script outputs on the real data.

2 Input Data and Target Model

2.1 Tensor Construction

The script reads `H:\HBN-EEG\derivatives\bandpower_8band_ordered_preproc_roi10\matlab\hbn_bandpo`
which contains:

- `EO_roi_power_uV2`,
- `EC_roi_power_uV2`,
- `age`,
- `roi_names`,
- `band_names`.

For subject i, let

$$X_i^{(EC)} \in \mathbb{R}^{10\times 8}, \qquad X_i^{(EO)} \in \mathbb{R}^{10\times 8}$$

denote the raw ROI-by-band power matrices in the EC and EO conditions.

The script first defines a positive floor

$$\texttt{rawFloor} = \max\left(\frac{1}{2}\min\{x : x > 0\},\, 10^{-12}\right),$$

where the minimum is taken over all positive entries in the EC and EO arrays. It then applies the transform

$$\widetilde{X}_i^{(EC)} = \log_{10}\big(\max(X_i^{(EC)}, \texttt{rawFloor})\big), \qquad \widetilde{X}_i^{(EO)} = \log_{10}\big(\max(X_i^{(EO)}, \texttt{rawFloor})\big).$$

The tensor covariate is constructed by stacking the two condition-specific matrices:

$$\mathcal{X}_i(:,:,1) = \widetilde{X}_i^{(EC)}, \qquad \mathcal{X}_i(:,:,2) = \widetilde{X}_i^{(EO)}.$$

Hence each subject contributes a third-order tensor

$$\mathcal{X}_i \in \mathbb{R}^{10\times 8\times 2},$$

with modes corresponding to:

- ROI,
- frequency band,
- condition (EC / EO).

The response is age:

$$y_i = \texttt{age}_i.$$

The model fitted by the script is a Tucker low-rank tensor regression model,

$$y_i \approx \langle \mathcal{X}_i, \mathcal{B} \rangle + \varepsilon_i,$$

where the coefficient tensor $\mathcal{B} \in \mathbb{R}^{10\times 8\times 2}$ is constrained to have low Tucker rank.

2.2 Candidate Rank Grid

The current default rank grid is

$$r_1 \in \{1,\dots,10\}, \qquad r_2 \in \{1,\dots,5\}, \qquad r_3 \in \{1,2\}.$$

Therefore the total number of candidate rank triples is

$$10 \times 5 \times 2 = 100.$$

The script constructs the full grid

$$\mathcal{R} = \{(r_1, r_2, r_3)\}$$

using `ndgrid`.

3 Outer Cross-Validation Procedure

3.1 Fold Assignment

Let N denote the sample size. The script fixes the random seed

$$\texttt{split_seed} = 20260321$$

and generates a random permutation π of the subject indices. The fold labels are then assigned according to

$$\texttt{foldId}(\pi_j) = (j-1) \bmod 5 + 1.$$

Thus each subject is assigned to one of five folds, and the same fold assignment is reused for all candidate ranks.

3.2 Evaluation of a Fixed Rank Triple

Fix a candidate Tucker rank

$$\mathbf{r} = (r_1, r_2, r_3).$$

For each fold $k = 1, \ldots, 5$:

- the training set is all observations with `foldId` $\neq k$,
- the validation set is all observations with `foldId` $= k$.

The script fits the model on the training set via

$$\texttt{fit_tucker_tensor_regression}(X_{\text{train}}, y_{\text{train}}, \mathbf{r}),$$

and obtains validation predictions via

$$\texttt{predict_tucker_tensor_regression}(\widehat{\mathcal{B}}, X_{\text{val}}).$$

The validation loss is

$$\text{MSE}_k(\mathbf{r}) = \frac{1}{n_k} \sum_{i \in \mathcal{V}_k} (y_i - \widehat{y}_i)^2.$$

The script also reports

$$R_k^2(\mathbf{r}) = 1 - \frac{\sum_{i \in \mathcal{V}_k} (y_i - \widehat{y}_i)^2}{\sum_{i \in \mathcal{V}_k} (y_i - \bar{y}_{\mathcal{V}_k})^2},$$

where $\bar{y}_{\mathcal{V}_k}$ is the mean of the validation responses within fold k.

3.3 Aggregation Across Folds

For each candidate rank $\mathbf{r}$, the script computes

$$\overline{\text{MSE}}(\mathbf{r}) = \frac{1}{5} \sum_{k=1}^{5} \text{MSE}_k(\mathbf{r}), \qquad \overline{R^2}(\mathbf{r}) = \frac{1}{5} \sum_{k=1}^{5} R_k^2(\mathbf{r}).$$

It also stores the empirical standard deviations of fold-wise MSE and fold-wise R^2.

3.4 Selection Rule

The summary table is sorted by the following priority:

1. `mean_val_mse` in ascending order,
2. `rank_sum`$=r_1 + r_2 + r_3$ in ascending order,
3. `rank1` in ascending order,
4. `rank2` in ascending order,
5. `rank3` in ascending order.

Hence the selected rank is the first row of the sorted summary table. Equivalently:

> Choose the rank triple with the smallest mean validation MSE; if there is a tie, prefer the lower total rank, and then the lexicographically smaller rank triple.

4 Internal Fitting Routine

4.1 Centering Within the Training Fold

Suppose the training set has size n. The fitting routine first computes

$$\bar{\mathcal{X}} = \frac{1}{n}\sum_{i=1}^{n} \mathcal{X}_i, \qquad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i,$$

and centers the training data:

$$\mathcal{X}_i^c = \mathcal{X}_i - \bar{\mathcal{X}}, \qquad y_i^c = y_i - \bar{y}.$$

The stored model retains `Xmean` and `ymean`. At prediction time, new samples are centered using the *training-fold mean*, not the validation-fold mean.

4.2 Vectorized Representation

The centered tensor observations are reshaped into a matrix

$$Z \in \mathbb{R}^{n\times(10\cdot 8\cdot 2)}.$$

The optimization target becomes

$$\min_{\mathcal{B}:\, \mathrm{rank}_{\mathrm{Tucker}}(\mathcal{B})\le(r_1,r_2,r_3)} \frac{1}{2n}\|Z\,\mathrm{vec}(\mathcal{B}) - y^c\|_2^2.$$

4.3 Initialization

The script first solves the unconstrained least-squares problem

$$\beta_0 = \arg\min_{\beta} \|Z\beta - y^c\|_2^2.$$

Implementation details:

- it uses `lsqminnorm(Z, ycentered)` if available;
- otherwise it falls back to `pinv(Z) * ycentered`.

The vector β_0 is reshaped into a tensor $\mathcal{B}^{(0)}$ and projected to the candidate Tucker rank.

4.4 Projection to Fixed Tucker Rank

Given a tensor $\mathcal{B}$, the script computes the leading left singular vectors of each mode unfolding:

$$U_1 = \mathrm{SVD}_{r_1}(B_{(1)}), \qquad U_2 = \mathrm{SVD}_{r_2}(B_{(2)}), \qquad U_3 = \mathrm{SVD}_{r_3}(B_{(3)}).$$

Then it forms the core tensor

$$\mathcal{G} = \mathcal{B} \times_1 U_1^\top \times_2 U_2^\top \times_3 U_3^\top$$

and reconstructs the projected tensor

$$\Pi_{\mathbf{r}}(\mathcal{B}) = \mathcal{G} \times_1 U_1 \times_2 U_2 \times_3 U_3.$$

This is the projection operator implemented in `project_tucker_tensor`.

4.5 Projected Gradient Descent

After initialization, the routine iterates

$$\mathcal{B}^{(t+\frac{1}{2})} = \mathcal{B}^{(t)} - \eta \nabla f(\mathcal{B}^{(t)}),$$

followed by projection,

$$\mathcal{B}^{(t+1)} = \Pi_{\mathbf{r}}\left(\mathcal{B}^{(t+\frac{1}{2})}\right),$$

where

$$f(\mathcal{B}) = \frac{1}{2n}\|Z\,\mathrm{vec}(\mathcal{B}) - y^c\|_2^2.$$

The gradient is

$$\nabla f(\mathcal{B}) = \mathrm{reshape}\left(\frac{1}{n} Z^\top (Z\,\mathrm{vec}(\mathcal{B}) - y^c)\right).$$

4.6 Step Size

Let $s_{\max}(Z)$ denote the largest singular value of the design matrix Z. The script defines

$$L = \max\left(\frac{s_{\max}(Z)^2}{n}, 10^{-8}\right), \qquad \eta = \frac{1}{L}.$$

Thus the step size is the inverse of a Lipschitz upper bound for the quadratic loss gradient.

4.7 Stopping Rule

At iteration t, the algorithm monitors:

$$\text{relative coefficient change} = \frac{\|\mathcal{B}^{(t+1)} - \mathcal{B}^{(t)}\|_F}{\max(\|\mathcal{B}^{(t)}\|_F, 10^{-12})},$$

$$\text{relative objective change} = \frac{|f(\mathcal{B}^{(t+1)}) - f(\mathcal{B}^{(t)})|}{\max(|f(\mathcal{B}^{(t)})|, 1)}.$$

The routine stops when either quantity is below `Tol`. The current rank-selection script uses

$$\texttt{MaxIter} = 4000, \qquad \texttt{Tol} = 10^{-6}.$$

4.8 Prediction Formula

Let $\widehat{\mathcal{B}}$ denote the fitted coefficient tensor, and let $\bar{\mathcal{X}}$ and $\bar{y}$ be the training-fold means stored in the model. For a new tensor covariate $\mathcal{X}$, the prediction is

$$\widehat{y} = \bar{y} + \langle \mathcal{X} - \bar{\mathcal{X}}, \widehat{\mathcal{B}} \rangle.$$

This ensures that training and validation use the same centering convention.

5 Default Output on the Real Data

The default output directory is `H:\HBN-EEG\derivatives\bandpower_8band_ordered_preproc_roi10\tucke`

The top 10 rows of the current `tucker_tensor_regression_cv_summary.csv` are:

r_1	r_2	r_3	$r_1 + r_2 + r_3$	mean val MSE	std val MSE	mean val R^2
6	2	2	10	7.9627	2.1841	0.2337
5	2	2	9	7.9708	2.1749	0.2330
3	2	2	7	8.0475	1.9957	0.2260
7	2	2	11	8.0672	2.1691	0.2236
4	2	2	8	8.0689	2.1515	0.2236
9	2	2	13	8.0698	2.1733	0.2237
8	2	2	12	8.0752	2.1699	0.2230
10	2	2	14	8.0790	2.1696	0.2227
3	2	1	6	8.2685	2.1458	0.2058
4	2	1	7	8.2881	2.1243	0.2037

Under the current default implementation, the selected rank is therefore

$$\widehat{\mathbf{r}}_{CV} = (6, 2, 2).$$

This is the rank selected by the script itself. A later inference analysis may override this and use $(4, 2, 2)$ for stability considerations, but that is a separate modeling choice and not part of the cross-validation routine described here.

6 Files Produced by the Script

The rank-selection script writes the following main outputs:

- `tucker_tensor_regression_cv_details.csv`: fold-by-fold validation results for every candidate rank;
- `tucker_tensor_regression_cv_summary.csv`: rank-level summary table after aggregating over folds;
- `tucker_tensor_regression_best_model.mat`: final model refit on the full sample using the selected rank;
- `tucker_tensor_regression_best_model_coefficients.csv`: elementwise export of the final coefficient tensor;
- `tucker_tensor_regression_cv_mean_mse.png`: CV heatmap over the rank grid;
- `tucker_tensor_regression_best_model_coefficients.png`: image of the final coefficient slices;
- `tucker_tensor_regression_summary.txt`: text summary of the selected rank and final fit.

7 Algorithm in Compact Form

The current implementation can be summarized as follows:

1. Read the ROI10 EEG data and construct $\mathcal{X}_i \in \mathbb{R}^{10\times 8\times 2}$ and y_i.
2. Generate a fixed 5-fold split using the preset random seed.
3. For each candidate Tucker rank $\mathbf{r} \in \mathcal{R}$:

   3.1. For each fold:

    3.1.1. Center the training tensors and training responses.
    3.1.2. Obtain an unconstrained least-squares initializer.
    3.1.3. Project the initializer to Tucker rank $\mathbf{r}$.
    3.1.4. Iterate projected gradient descent until convergence.
    3.1.5. Predict the validation fold using the training-fold centering constants.
    3.1.6. Record validation MSE and validation R^2.
  3.2. Aggregate the 5 validation losses for this rank.

4. Sort all candidate ranks by mean validation MSE, then by lower total rank.

5. Select the first rank in that sorted table.

6. Refit the model on the full sample using the selected rank and export the results.

8 Recommended One-Sentence Description

If a concise description is needed in a paper or report, the algorithm can be summarized as:

> We form a $10 \times 8 \times 2$ tensor covariate from the log-transformed EC and EO ROI-band power matrices, fit Tucker low-rank tensor regression models over a prespecified rank grid, and select the Tucker rank by 5-fold cross-validation using the mean validation MSE, with ties broken in favor of lower total rank.